

Assignment 1

Guidelines:

1. Logistics. Your solution must be submitted as a *typed* PDF document. Scanned handwritten solutions, solutions in any other format, or unreadable solutions will not be accepted. You are encouraged to use L^AT_EX.

Submit your solution in Crowdmark by **Tuesday, Feb 25, 23:59**. You are strongly encouraged to submit the solution at least *several days in advance* – last-minute technical problems in your systems and/or in Crowdmark systems will not grant any extension to the deadline.

2. Answering questions. In all assignments and exams in this course, unless specifically and explicitly stated otherwise:

- Establish correctness for all of your answers using rigorous proofs. Your solution will be marked based on its completeness and correctness as well as on the correctness, clarity and precision of your proofs.
- Describe algorithms in high-level pseudocode (at the same level of abstraction used in class, e.g. for dynamic programming). Do not provide low-level executable code. Also, if your code is overly abstract and does not sufficiently specify relevant parts (e.g., it is not obvious how to efficiently implement parts), you may lose marks.
- When you are asked to prove an upper bound on the running time of an algorithm, the intention is to prove the tightest upper bound that you can. Non-tight upper bounds may receive partial (or no) marks.

3. Academic integrity. Assignments are to be solved individually, and according to the course integrity policy (outlined in the syllabus) and the Code of Behaviour on Academic Matters. Here is a reminder of one of the relevant parts from the syllabus:

Large assignments should be solved individually, and written by yourself, without imitating online texts and without making use of generative AI (or any other software or automated tool). Any submitted assignment that is suspiciously similar to another assignment or to online texts will be automatically investigated as an integrity violation.

You are encouraged to ask questions on Piazza, without directly asking how to solve a problem or revealing the solution. You may not consult any other resources except your class notes, your textbook, Piazza, and assigned readings. Consulting any other resource or collaborating with other students is a violation of the academic integrity policy.

Question 1. In class we saw a greedy strategy for solving the interval scheduling problem. Describe a different greedy strategy that does *not* correctly solve this problem, and prove that for any integer n there is an input (i.e., a set of n intervals) for which the strategy produces an incorrect output. (5pts.)

(Note: Your answer must explicitly present an input for any $n \in \mathbb{N}$, and prove that the algorithm is incorrect on this input.)

Answer 1. Consider the following algorithm:

```
sort the intervals by descending length
let  $T = \emptyset$ 
for  $i = 1, \dots, n$ :
    if  $I_i$  is compatible with  $T$ , add  $I_i$  to  $T$ 
return  $T$ 
```

For every $n \in \mathbb{N}$, consider the set of intervals where $I_1 = [0, n]$ and for each $i \in \{2, \dots, n\}$ we have $I_i = [i - 2, i - 1]$.¹ Since $|I_1| = n$ and $|I_i| < 1$ for all $i \in \{2, \dots, n\}$, the algorithm always inspects I_1 first, and adds it to T . But $I_1 \cap I_i \neq \emptyset$ for all $i \in \{2, \dots, n\}$, and hence the algorithm will never add another interval to T . Thus, the algorithm outputs $T = \{I_1\}$, whereas an optimal solution has at least two intervals, say I_2 and I_3 , which are disjoint.²

¹Formally, in class we required the intervals to be closed intervals. To meet this we may define $I_i = [i - 2, i - 1 - \epsilon]$ for some small ϵ (say, $\epsilon = 10^{-6}$).

²Indeed, the optimal solution has the $n - 1$ intervals $[0, 1), [1, 2), \dots, [n - 2, n - 1)$, but you are not required to prove this – you only need to prove that an optimal solution has more intervals than what the algorithm outputs.

Question 2. Given an n -vertex undirected graph $G = (V, E)$, a vertex cover is a set of vertices $S \subseteq V$ such that every edge $e \in E$ touches some $s \in S$. Our goal is to find a vertex cover of minimal size. Consider the following greedy algorithm:

```

let  $S = \emptyset, E' = E$ 
while  $E' \neq \emptyset$ :
    let  $e = (u, v)$  be an arbitrary edge in  $E'$ 
    add  $u$  and  $v$  to  $S$ 
    remove from  $E'$  all edges incident to  $u$  and to  $v$ 
return  $S$ 

```

Let $OPT(G)$ be the size of the minimal vertex cover for a graph G . Either prove that for every graph G , the greedy algorithm above outputs a vertex cover of size at most $2 \cdot OPT(G)$; or show that for every integer n there is a graph G on which this algorithm outputs a vertex cover of size more than $2 \cdot OPT(G)$. (10pts.)

Answer 2. We prove that for every graph G the algorithm returns a vertex cover of size at most $2 \cdot OPT(G)$.

Let G be any graph, and let $\tilde{E}$ be the set of edges chosen by the algorithm during its execution on G . Since the algorithm adds both endpoints of each $e \in \tilde{E}$ to S , we have that $S = \cup_{e=(u,v) \in \tilde{E}} \{u, v\}$, and hence $|S| \leq 2|\tilde{E}|$. Moreover, there do not exist two different edges in $\tilde{E}$ that touch a shared vertex, since after adding $\{u, v\}$ to S , the algorithm removes from E' all edges incident to $\{u, v\}$ (and thus the algorithm will not attempt to add one of these vertices again to S).

Now, let $S^* \subseteq V$ be a vertex cover for G of optimal size. Note that S^* must contain at least one vertex from each $e \in \tilde{E}$ (by the definition of a vertex cover). But since no two edges in $\tilde{E}$ touch a shared vertex, S^* must contain at least $|\tilde{E}|$ vertices. Therefore $|S^*| \geq |\tilde{E}| \geq |S|/2$.

Question 3. Consider an $n \times n$ matrix. A generalized diagonal is a sequence of n distinct indices $j_1, \dots, j_n \in [n]$ (i.e., $j_i \neq j_{i'}$ for all $i \neq i'$) such that in each row $i \in [n]$, the entry at column j_i is 1.³

Given an $n \times n$ matrix, we want to find a generalized diagonal. Consider the following greedy algorithm:

```

for  $i = 1, \dots, n$ :
    let  $j_i$  be the left-most index such that the  $j_i^{th}$  entry in the  $i^{th}$  row is 1
    and  $j_i \neq j_k$  for all  $k \in \{0, \dots, i-1\}$ , or return "fail" if no such  $j_i$  exists
return  $j_1, \dots, j_n$ 

```

Prove that this algorithm is correct, or that for every integer n there is an input matrix on which this algorithm fails. (5pts.)

Answer 3. For every n , consider a matrix where the first row has 1 in the first two entries and then all zeroes; the second row has 1 in the first entry and then all zeroes; and for each row $i \in \{3, \dots, n\}$, there is only a single 1, appearing in the i^{th} entry (i.e., along the standard diagonal). Note that this matrix has a generalized diagonal with entries $j_1 = 2, j_2 = 1$ and $j_i = i$ for all $i \in [n]$.

The algorithm would start with the first row, and let $j_1 = 1$. However, then in row 2 there is no entry with 1 except for the first entry, which is already taken by j_1 , and the algorithm would return "fail". This is incorrect.

³To develop intuition, Note that when the matrix has 1's across its standard diagonal, we can take $j_i = i$ for all $i \in [n]$, in which case the generalized diagonal is just the standard diagonal. Also note that some matrices don't have a generalized diagonal, for example the all-0's matrix.

Question 4. An LLM suggested an algorithm to multiply two integers, each represented by n bits, using recursion. The algorithm makes twelve recursive calls to multiplication problems on integers with $n/5$ bits, and then takes $O(n^{3/2})$ time to combine the results. The LLM did not provide further details, but claimed that this is the fastest known algorithm solving the problem.

What is the running time of the algorithm? Justify your answer using the Master theorem.⁴ Is this algorithm faster than Karatsuba's algorithm? Is it faster than the best known algorithm for this problem? (5pts.)

Answer 4. We use the Master theorem with parameters $a = 12$ and $b = 5$ and $d = 1.5$. Note that $c = \log_b(a) \in (1.54, 1.55)$, and so $c > d$. Therefore the running time is $O(n^c) \approx n^{1.543\dots}$. This is faster than Karatsuba's algorithm, which runs in time $O(n^{\log_2(3)}) \approx n^{1.584\dots}$, but slower than the best known algorithm, which runs in time $O(n \cdot \log n)$.

⁴When analyzing the hallucinated algorithm, you may assume for simplicity that n is a power of 5.

Question 5. Prove that the Fast Fourier Transform algorithm runs in time $O(n \cdot \log n)$ on every integer $n \geq 2$ that is a power of two. The proof should use induction, and not use the Master theorem. (10pts.)

Answer 5. We prove that there exists a constant $c > 1$ such that the algorithm runs in time at most $c \cdot n \cdot \log(n)$ on inputs of length $n \geq 2$. The proof uses induction on n , and a suitable value for c will be specified during the proof.

The base case $n = 2$, the algorithm uses $k_2 = O(1)$ many operations, and the bound holds as long as $2c \geq k_2$ (so it will suffice to take $c \geq k_2/2$).

For the induction step, assume that the claim is true for input length $n/2$. Excluding the two recursive calls, at input length n the algorithm runs in linear time, or in other words in time $k \cdot n$ for some constant k (note that k does *not* depend on n or on our choice of c). By the induction hypothesis, each recursive call runs in time at most $c \cdot (n/2) \cdot \log(n/2)$. Thus, overall, the running time of the algorithm is at most

$$\begin{aligned} 2 \cdot (c \cdot (n/2) \cdot \log(n/2)) + k \cdot n &= c \cdot n \cdot (\log(n) - 1) + k \cdot n \\ &= c \cdot n \cdot \log(n) - (c - k) \cdot n, \end{aligned}$$

and the bound holds as long as $c \geq k$.

Overall, both the base case and the induction step hold if we choose c to be any constant larger than $\max\{k, k_2/2\}$.

Question 6. An $n \times n$ matrix A is a Toeplitz matrix if for each diagonal of A , all entries along the diagonal have the same value. Here is a visual depiction:

$$\begin{pmatrix} a_{n-1} & a_{n-2} & a_{n-3} & \dots & a_1 & a_0 \\ a_n & a_{n-1} & a_{n-2} & \dots & a_2 & a_1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots \\ a_{2n-3} & \dots & \dots & \dots & a_{n-1} & a_{n-2} \\ a_{2n-2} & a_{2n-3} & \dots & \dots & a_n & a_{n-1} \end{pmatrix}$$

Explain how an $n \times n$ Toeplitz matrix can be represented using $2n - 1$ elements. Then describe an algorithm that gets such a representation along with a vector v of length n , and outputs $A \cdot v$. Prove an upper bound on the running time of your algorithm. (15pts.)

(Note: A quadratic-time algorithm isn't good enough - design an algorithm running in time $O(n \cdot \log n)$.)

(Hint: To develop intuition, use notation for the entries of A as in the visual depiction, denote $v = v_0, v_1, \dots, v_{n-1}$, and write down the elements of $A \cdot v$ explicitly, then stare at them for a while. Now consider two polynomials p and q of degree at most $2n - 2$ such that the coefficients of p are $a_0, \dots, a_{2n-2}$ and the coefficients of q are $v_0, \dots, v_{n-1}, 0, \dots, 0$, and the entries of $A \cdot v$ are coefficients of the multiplication $p \cdot q$.)

Answer 6. We use the notation $a_0, \dots, a_{2n-2}$ for the entries of A , as in the visual depiction. A Toeplitz matrix A can then be represented by the vector $a_0, \dots, a_{2n-2}$. For $i \in \{0, \dots, n-1\}$, the i^{th} element of $A \cdot v$ is

$$(A \cdot v)_i = \sum_{k=0}^{n-1} a_{n-1+i-k} \cdot v_k.$$

Let $p(x) = \sum_{i=0}^{2n-2} a_i \cdot x^i$ and let $q(x) = \sum_{i=0}^{n-1} v_i \cdot x^i$. Observe that for $i \in \{0, \dots, 3n-3\}$, the coefficient of x^i in $(p \cdot q)$ is $\sum_{k=0}^i v_k \cdot a_{i-k}$.⁵ In particular, for $i \in \{0, \dots, n-1\}$, the coefficient of x^{n-1+i} in $p \cdot q$ is

$$\sum_{k=0}^{n-1+i} v_k \cdot a_{n-1+i-k} = \sum_{k=0}^{n-1} v_k \cdot a_{n-1+i-k} = (A \cdot v)_i,$$

where the first inequality is since $v_k = 0$ for all $k > n-1$.

Given $a_0, \dots, a_{2n-2}$ and $v_0, \dots, v_{n-1}$, we can compute the coefficients of the two polynomials p and q in time $O(n)$. We then use the method learned in class to obtain the coefficients of the multiplication $p \cdot q$, which may be of degree $3n-1$, in time $O(n \cdot \log(n))$. (Specifically, use FFT to convert the coefficient-representation of p and q

⁵For notational convenience, in the summation expression, we interpret entries v_k with $k > n-1$ as zero, and similarly a_{i-k} with $i-k > 2n-2$ as zero.

into evaluations representation (i.e., a representation over the Fourier basis), compute $p \cdot q$ in the evaluations representation, and use FFT again on the auxiliary polynomial shown in class to interpolate and compute the coefficients of $p \cdot q$). We then output the entries of $A \cdot v$ where the i^{th} entry is the coefficient of x^{n-1+i} in $p \cdot q$; this is done, again, in linear time, by copying the coefficients.

Question 7. A person is climbing n stairs, and at each step they can climb either one stair or two stairs. The input to the problem is a sequence of real-valued costs $c_{1,1}, c_{1,2}, c_{2,1}, c_{2,2}, \dots, c_{n-2,1}, c_{n-2,2}, c_{n-1,1}$, where $c_{i,b}$ represents the cost of starting from stair $i \in [n]$ and climbing $b \in \{1, 2\}$ stairs, and costs may be negative. The total cost of a path from 1 to n is the sum of costs of steps along the way.

Write pseudocode for an algorithm that gets the sequence of costs and finds the minimal total cost of climbing n stairs. Prove the algorithm's correctness, and prove an upper-bound on its running time. (10pts.)

(Note: In this question and all other questions, if you are relying on dynamic programming and using a formula expressing the solution to a sub-problem in terms of solutions to other sub-problems, you must prove that your formula is correct.)

Answer 7. For $i \geq 1$, let $w^*(i)$ be the optimal cost of going from stair 1 to stair i . Note that $w^*(1) = 0$ and that $w^*(2) = c_{1,1}$ (since there is only one way to climb from stair 1 to stair 2). We rely on the following recursive formula:

Lemma 1. For every $i \geq 3$, we have

$$w^*(i) = \max \{w^*(i-1) + c_{i-1,1}, w^*(i-2) + c_{i-2,2}\}$$

Proof. Let S^* be an optimal path from stair 1 to stair i . Either the path passes through stair $i-1$ or it doesn't.

- In the first case, the path consists of a subpath S from 1 to $i-1$ and a step from $i-1$ to i of cost $c_{i-1,1}$. Note that the cost of S must be $w^*(i-1)$, otherwise we could replace S by a subpath of lower cost c , add a step from $i-1$ to i , and obtain a path of cost $c + c_{i-1,1}$, which is lower than S^* – a contradiction. Hence, in the first case, the cost of the optimal path S^* is $w^*(i-1) + c_{i-1,1}$.
- In the second case, the path S^* must first reach $i-2$ and then take a double step to i of cost $c_{i-2,2}$ (since we are only allowed one step or two steps, and S^* does not pass through $i-1$). By an exchange argument as above, the path S in S^* from 1 to $i-2$ must be of cost $w^*(i-2)$, and hence in the second case, the total cost of the optimal path S^* is $w^*(i-2) + c_{i-2,2}$.

■

The algorithm.

```

1: Input:  $c_{1,1}, c_{1,2}, \dots, c_{n-1,1}$ 
2: initialize  $opt$  as an empty array with indices  $1, \dots, n$ 
3:  $opt[1] \leftarrow 0$ 
4:  $opt[2] \leftarrow c_{1,1}$ 
5: for  $i = 3, \dots, n$  do
6:    $opt[i] = \max \{opt[i-1] + c_{i-1,1}, opt[i-2] + c_{i-2,2}\}$ 
7: return  $opt[n]$ 

```

Analysis. The algorithm runs in time $O(n)$, since there is one “for” loop with $n - 2$ iterations, and each iteration involves $O(1)$ operations.

For correctness, we argue that for every $i = 3, \dots, n$, after the iteration of the “for” loop with value i ends, we have $opt[j] = w^*(j)$ for every $j \leq i$. (This is enough, since at the end the algorithm returns $opt[n] = w^*(n)$.) This is proved by induction: The base case $i = 3$ is by the lemma, line 6, and since $opt[1]$ and $opt[2]$ are defined as $w^*(1)$ and $w^*(2)$ in lines 3-4; and the inductive step follows by the lemma and line 6.

Question 8. A farmer has n plots of land, and in each plot they can either grow corn or grow pumpkins. They can also leave a plot empty. The only constraint is that no two consecutive plots i and $i + 1$ can grow the same crop.

The input to the problem is two sequences of positive real numbers $\{c_1, \dots, c_n\}$ and $\{p_1, \dots, p_n\}$ where c_i and p_i are the profits from planting corn and pumpkin at plot i , respectively. The total profit is the sum of profits in all plots. An empty plot yields zero profit.

Write pseudocode for an algorithm that gets two sequences of profits and finds the maximum total profit that the farmer can get. Prove the algorithm's correctness, and prove an upper-bound on its running time. (10pts.)

Answer 8. Let $c^*(i)$ be the optimal profit from plots $1, \dots, i$ when plot i is planted with corn, and let $p^*(i), e^*(i)$ be defined analogously for when the i^{th} plot is planted with pumpkin and for when the i^{th} plot is empty, respectively.

By definition, we have $c^*(1) = c_1, p^*(1) = p_1, e^*(1) = 0$. We rely on the following recursive formula:

Lemma 2. For $i \in \{2, \dots, n\}$:

$$\begin{aligned} c^*(i) &= \max \{e^*(i-1) + c_i, p^*(i-1) + c_i\} \\ p^*(i) &= \max \{e^*(i-1) + p_i, c^*(i-1) + p_i\} \\ e^*(i) &= \max \{c^*(i-1), p^*(i-1), e^*(i-1)\} \end{aligned}$$

Proof. Let S^* be an optimal plan for planting plots $1, \dots, i$ with corn in plot i . Since corn cannot be planted in plot $i-1$ in S^* , either plot $i-1$ in S^* is planted with pumpkin or it is left empty. In the first case, the profit of S^* is $p^*(i-1) + c_i$, by an exchange argument (if there is a way to plant in plots $1, \dots, i-1$ with plot $i-1$ being pumpkin such that this plan has profit more than $p^*(i-1)$, we use it along with corn in plot i to obtain a plan for plots $1, \dots, i$ with corn in plot i and profit exceeding that of S^* , a contradiction). In the second case, the profit of S^* is $e^*(i-1) + c_i$, again by an exchange argument (the details of the exchange argument are essentially identical).

The argument for $p^*(i)$ is identical to the argument for $c^*(i)$, replacing “corn” with “pumpkin”: Considering an optimal plan S^* with pumpkin in plot i , we use a case analysis according to whether plot $i-1$ is planted with corn or left empty (it cannot have pumpkin), and use exchange arguments to argue that in the first case the profit of S^* is $c^*(i-1) + p_i$ and in the second case it is $e^*(i-1) + p_i$.

The case of $e^*(i)$ is different. Let S^* be an optimal plan with plot i left empty. Now there are three cases: Plot i may have corn, may have pumpkin, or may be empty. The cases are handled identically to the arguments above: using exchange arguments, we argue that the value of S^* must be $c^*(i-1)$, or $p^*(i-1)$, or $e^*(i-1)$, respectively. ■

Remark 3. In Lemma 2, it is also true that $e^*(i) = \max \{c^*(i-1), p^*(i-1)\}$ (i.e., without considering the case of $e^*(i-1)$). However, this fact isn't obvious, and requires a complete and convincing proof (if you want to rely on it). One way to prove this fact is to show by induction that for every $i \geq 1$ it holds that $e^*(i) \leq \max \{c^*(i), p^*(i)\}$.

The algorithm.

```

1: Input:  $c_1, \dots, c_n, p_1, \dots, p_n$ 
2: initialize corn, pumpkin, empty as empty arrays, each with indices  $1, \dots, n$ 
3: corn[1]  $\leftarrow c_1$ 
4: pumpkin[1]  $\leftarrow p_1$ 
5: empty[1]  $\leftarrow 0$ 
6: for  $i = 2, \dots, n$  do
7:   corn[ $i$ ] =  $\max \{ \textit{empty}[i-1] + c_i, \textit{pumpkin}[i-1] + c_i \}$ 
8:   pumpkin[ $i$ ] =  $\max \{ \textit{empty}[i-1] + p_i, \textit{corn}[i-1] + p_i \}$ 
9:   empty[ $i$ ] =  $\max \{ \textit{corn}[i-1], \textit{pumpkin}[i-1], \textit{empty}[i-1] \}$ 
10: return  $\max \{ \textit{corn}[n], \textit{pumpkin}[n], \textit{empty}[n] \}$ 

```

Analysis. The algorithm runs in time $O(n)$, since there is one “for” loop with $n - 1$ iterations, and each iteration involves $O(1)$ operations.

For correctness, we argue that for every $i = 2, \dots, n$, after the iteration of the “for” loop with value i ends, we have $\textit{corn}[i] = c^*(i)$ and $\textit{pumpkin}[i] = p^*(i)$ and $\textit{empty}[i] = e^*(i)$. Proving this claim will suffice for the proof, because the algorithm returns the maximum value among $\textit{corn}[n] = c^*(n)$, $\textit{pumpkin}[n] = p^*(n)$, $\textit{empty}[n] = e^*(n)$, and any optimal solution S^* must have one of the three in plot n .

The proof of the claim is by induction. The base case of $i = 2$ follows by the lemma, lines 7–9, and since $\textit{corn}[1]$, $\textit{pumpkin}[1]$, $\textit{empty}[1]$ are defined as $c^*(1)$, $p^*(1)$, $e^*(1)$ (respectively) in lines 3–5. The induction step follows from the lemma and lines 7–9.

Question 9. In this problem the input is a string $x_1, \dots, x_n$ where $x_i \in \{a, b, c\}$ for each $i \in [n]$ (i.e., $x_1, \dots, x_n$ is a string of alphabet letters that can be either “a”, “b”, or “c”). For each consecutive substring $x_j, x_{j+1}, \dots, x_{j+k}$ (where $j \in [n]$ and $k \in 0, \dots, n-j$), define the discrepancy of the substring as the number of times “b” appears minus the number of times “a” appears; that is:

$$|\{m \in j, \dots, j+k : x_m = b\}| - |\{m \in j, \dots, j+k : x_m = a\}|$$

Write pseudocode for an algorithm that gets $x_1, \dots, x_n$ and finds the maximal discrepancy among all consecutive substrings. Prove the algorithm’s correctness, and prove an upper-bound on its running time. (15pts.)

(Note: A quadratic-time algorithm isn’t good enough – design a faster algorithm.)

Answer 9. Let $OPT^*(i)$ be the maximal discrepancy among all consecutive substrings

that start with i . Fixing any string x , for every $i \in [n]$ let $\sigma_i = \begin{cases} -1 & x_i = a \\ 1 & x_i = b \\ 0 & x_i = c \end{cases}$.

By definition, $OPT^*(n) = \sigma_n$. We rely on the following recursive formula:

Lemma 4. For every $i \in [n-1]$,

$$OPT^*(i) = \max \{\sigma_i, OPT^*(i+1) + \sigma_i\}$$

Proof. Let $S^*(i)$ be a consecutive substring starting at i that attains the maximal discrepancy among all consecutive substrings starting at i . Either the substring $S^*(i)$ is of length 1, or it is of length more than one. In the first case the discrepancy of $S^*(i)$ is σ_i , by definition. In the second case, consider the substring S of $S^*(i)$ we get by removing x_i . The substring S must be of discrepancy $OPT^*(i+1)$ (by an exchange argument – otherwise, we could replace it by a substring of larger discrepancy, and after adding x_i we’d get a substring starting at i with larger discrepancy than that of $S^*(i)$). Thus, in the second case, the discrepancy of $S^*(i)$ is $\sigma_i + OPT^*(i+1)$. ■

The algorithm.

```

1: procedure VAL(z)
2:   if z = a then
3:     return -1
4:   else if z = b then
5:     return 1
6:   else
7:     return 0

```

```

1: Input:  $x_1, \dots, x_n$ 

```

```

2: initialize  $opt$  as an empty array with indices  $1, \dots, n$ 
3:  $opt[n] \leftarrow Val(x_n)$ 
4:  $max \leftarrow opt[n]$ 
5: for  $i = n - 1, \dots, 1$  do
6:   if  $opt[i + 1] > 0$  then
7:      $opt[i] \leftarrow Val(x_i) + opt[i + 1]$ 
8:   else
9:      $opt[i] \leftarrow Val(x_i)$ 
10:  if  $opt[i] > max$  then
11:     $max \leftarrow opt[i]$ 
12: return  $max$ 

```

Analysis. The algorithm runs in time $O(n)$, since there is one “for” loop with $n - 1$ iterations, and each iteration involves $O(1)$ operations.

For correctness, by definition $Val(x_i) = \sigma_i$ for all $i \in [n]$. We then argue that:

Claim 5. *For every $i = n - 1, \dots, 1$, after the iteration of the “for” loop with value i ends, we have $opt[j] = OPT^*(j)$ for all $j \in \{i, \dots, n\}$, and $max = \max_{j \geq i} \{OPT^*(j)\}$.*

Proof. The proof is by induction on $i = n - 1, \dots, 1$. For the base case of $i = n - 1$, note that $opt[n] = OPT^*(n)$ after line 3, by the definition of $OPT^*(n)$ and since $Val(n) = \sigma_n$. The fact that $opt[n - 1] = OPT^*(n - 1)$ follows from Lemma 4 and from lines 6–9. The fact that $max = \max_{j \geq n-1} \{OPT^*(j)\}$ follows from lines 4 and 10-11 (where the latter have only been executed once in the base case).

For the induction step, let $i \in \{1, \dots, n - 2\}$. We know that after the iteration of the “for” loop with value $i + 1$, we had $opt[j] = OPT^*(j)$ for all $j \in i + 1, \dots, n$ and $max = \max_{j \geq i+1} OPT^*(j)$. The fact that after the iteration of the loop with value i we have $opt[i] = OPT^*(i)$ follows by lines 6–9 and Lemma 4 and by the induction hypothesis. The fact that $opt[j] = OPT^*(j)$ for $j \in \{i + 1, \dots, n\}$ follows from the induction hypothesis and since $opt[j]$ for $j > i$ isn’t modified in the iteration of the “for” loop with value i . The fact that $max = \max_{j \geq i} \{OPT^*(j)\}$ follows from lines 10-11 (relying on the fact that the value of $opt[i] = OPT^*(i)$ has already been determined by lines 6–9 when reaching lines 10-11). ■

Now observe that the maximal discrepancy of any substring is $\max_{i \in [n]} \{OPT^*(i)\}$, since any substring has to start at some $i \in [n]$. By the claim above, this is the value returned by the algorithm at the end of its execution.

Question 10. In this problem we are given $x_1, \dots, x_n, k \in \mathbb{Z}_d$. Our goal is to decide whether it is possible to insert $+$ and $-$ symbols between the numbers $x_1, \dots, x_n$ so that the resulting expression evaluates to k over $\mathbb{Z}_d$.⁶

Write pseudocode for an algorithm that gets $x_1, \dots, x_n, k \in \mathbb{Z}_d$, runs in time $O(n \cdot d)$, and outputs “yes” if it is possible to add $+$ and $-$ symbols so that the expression evaluates to k , and outputs “no” if it is impossible. Prove the algorithm’s correctness, and prove the required upper-bound on its running time. (15pts.)

Answer 10. Fix an input $x_1, \dots, x_n, k$. For every $i \in [n]$ and $j \in [d]$, let $T(i, j) \in \{0, 1\}$ equal 1 if and only if it is possible to insert $\pm$ signs between the first i integers so that the expression evaluates to j modulo d . Note that $T(1, j) = 1 \iff x_1 = j$. We rely on the following recursive formula:

Lemma 6. For $i > 1$ we have:

$$T(i, j) = 1 \iff T(i-1, j - x_i \bmod d) = 1 \vee T(i-1, j + x_i \bmod d) = 1$$

Proof. For the $\Leftarrow$ direction, if there is a way to add $\pm$ signs between $x_1, \dots, x_{i-1}$ so that the sum equals $j - x_i$ modulo d , we can add these signs as well as a $+$ sign before x_i so that the sum of $x_1, \dots, x_i$ equals j modulo d . Similarly, if there’s a way to add $\pm$ signs between $x_1, \dots, x_{i-1}$ so that the sum equals $j + x_i$ modulo d , we can add a $-$ sign and the summation of $x_1, \dots, x_i$ will equal j modulo d .

For the $\Rightarrow$ direction, assume that there’s a way to add $\pm$ signs between $x_1, \dots, x_i$ so that the sum equals j modulo d . Fixing any such way, either the last symbol is $+$ or the last symbol is $-$. In the first case, the sum of $x_1, \dots, x_{i-1}$ with these signs must equal $j - x_i$ modulo d , so $T(i-1, j - x_i \bmod d) = 1$; and in the second case, the sum must equal $j + x_i$ modulo d , so $T(i-1, j + x_i \bmod d) = 1$. ■

The algorithm. All the arithmetic in the algorithm below is executed in $\mathbb{Z}_d$.

```

1: Input:  $x_1, \dots, x_n, k$ 
2: initialize  $eq$  as an empty  $n \times d$  array
3: for  $j = 1, \dots, d$  do
4:    $eq[1, j] \leftarrow 0$ 
5:  $eq[1, x_1] \leftarrow 1$ 
6: for  $i = 2, \dots, n$  do
7:   for  $j = 1, \dots, d$  do
8:     if  $eq[i-1, j - x_i] = 1$  then
9:        $eq[i, j] = 1$ 
10:    else if  $eq[i-1, j + x_i] = 1$  then
11:       $eq[i, j] = 1$ 

```

⁶Recall what it means to work over $\mathbb{Z}_d$: The elements $x_1, \dots, x_n, k$ are numbers in $\{0, \dots, d-1\}$, and all arithmetic operations are performed modulo d .

```
12:         else
13:              $eq[i, j] = 0$ 
14: return  $eq[n, k]$ 
```

Analysis. The algorithm runs in time $O(n \cdot d)$, since the “for” loop in line 3 runs in time $O(d)$ and lines 6–7 are nested loops with $n - 1$ and d iterations, respectively, where the inner computation in lines 8–13 uses $O(1)$ operations.

For correctness, we argue that for each $i \in \{2, \dots, n\}$, after the i^{th} iteration of the loop in line 6, for every $j \in [d]$ it holds that $eq[i, j] = T(i, j)$. (The proof is by induction. We first observe that $eq[1, j] = T(1, j)$ for all $j \in [d]$, by definition of $T(1, j)$ and lines 3–4. Then, both the base case and the induction step follow from lines 7–13 and from the lemma.) Finally, the algorithm outputs $eq[n, k] = T(n, k)$, which (by definition) equals 1 iff it’s possible to insert $\pm$ signs between the n integers so that the expression evaluates to k modulo d .

Remark 7. When proving the induction step, we rely on the fact that the lemma proved an “if and only if” statement. Had the lemma only proved the $\implies$ direction, it could have been that (for example) $T(i, j) = 1$ only if the AND of the two conditions on the RHS is true. Conversely, had the lemma only proved the $\impliedby$ direction, it could have been that $T(i, j) = 1$ also in additional situations (i.e., in more situations than the one specified in the RHS).